

封装成独立 Go Package

好，下面我给你 **封装成独立 Go Package**，结构清晰、接口干净、可直接 `go get / import`。

yamlpatch — 通用 YAML Patch 引擎 (Go Package)

一、包结构

```
yamlpatch/  
├─ go.mod  
├─ go.sum  
├─ types.go      # 数据结构  
├─ patch.go      # Patch 主逻辑  
├─ merge.go      # deepMerge  
├─ diff.go       # diff / dry-run  
└─ utils.go      # 工具函数
```

二、go.mod

```
module github.com/your-org/yamlpatch  
  
go 1.21  
  
require gopkg.in/yaml.v3 v3.0.1
```

三、types.go (公共 API)

```
package yamlpatch  
  
import "gopkg.in/yaml.v3"  
  
// ResourcePatchSpec 对应 ResourcePatch CR 的 spec  
type ResourcePatchSpec struct {
```

```

        Namespace string      `yaml:"namespace"`
        TargetRef TargetRef `yaml:"targetRef"`
        Patches []PatchOp `yaml:"patches"`
    }

    type TargetRef struct {
        APIVersion string `yaml:"apiVersion"`
        Kind        string `yaml:"kind"`
        Name        string `yaml:"name"`
    }

    // PatchOp 单条 patch 操作
    type PatchOp struct {
        Ope      string      `yaml:"ope"`
        PathKey  string      `yaml:"pathKey"`
        ByKey    string      `yaml:"byKey,omitempty"`
        ItemOps  string      `yaml:"itemOps,omitempty"`
        MixedAr  bool        `yaml:"mixedAr,omitempty"`
        Old      interface{} `yaml:"old,omitempty"`
        Value    interface{} `yaml:"value"`
    }

    // Diff 表示一次变更
    type Diff struct {
        Path string
        Old  interface{}
        New  interface{}
    }
}

```

四、patch.go (Patch 主入口)

```

package yamlpatch

import (
    "fmt"
    "strings"
)

// Apply 对 obj 原地应用 patches
func Apply(obj map[string]interface{}, patches []PatchOp) error {
    for _, p := range patches {
        if err := applyOne(obj, p); err != nil {
            return err
        }
    }
}

```

```

        }
    }
    return nil
}

func applyOne(obj map[string]interface{}, p PatchOp) error {
    parts := strings.Split(strings.Trim(p.PathKey, "/"), "/")
    parent, lastPart, err := resolveParent(obj, parts)
    if err != nil {
        return err
    }

    switch p.Ope {
    case "merge":
        return applyMerge(parent, lastPart, p)
    case "replace":
        return applyReplace(parent, lastPart, p)
    case "delete":
        return applyDelete(parent, lastPart, p)
    default:
        return fmt.Errorf("unknown ope: %s", p.Ope)
    }
}

func resolveParent(obj map[string]interface{}, parts []string) (map[string]interface{},
string, error) {
    cur := obj
    for i, p := range parts[:len(parts)-1] {
        v, ok := cur[p]
        if !ok {
            return nil, "", fmt.Errorf("path not found: %s", strings.Join(p
arts[:i+1], "/"))
        }
        m, ok := v.(map[string]interface{})
        if !ok {
            return nil, "", fmt.Errorf("intermediate node is not map")
        }
        cur = m
    }
    return cur, parts[len(parts)-1], nil
}

```

五、merge.go (deepMerge + 数组策略)

```

package yamlpatch

import (
    "fmt"
    "reflect"
)

func applyMerge(parent map[string]interface{}, key string, p PatchOp) error {
    existing, _ := parent[key]

    // struct 数组
    if p.ByKey != "" {
        return mergeStructArray(existing, p)
    }

    // scalar 数组
    if p.ItemOps != "" {
        return applyItemOps(existing, p)
    }

    // map 递归 merge
    if isMap(existing) && isMap(p.Value) {
        deepMerge(existing.(map[string]interface{}), p.Value.(map[string]interf
ace{}))
        return nil
    }

    // scalar 覆盖
    parent[key] = p.Value
    return nil
}

// ---- struct 数组 (byKey) ----
func mergeStructArray(existing interface{}, p PatchOp) error {
    arr, ok := existing.([]interface{})
    if !ok {
        return fmt.Errorf("byKey requires array")
    }
    valArr, ok := p.Value.([]interface{})
    if !ok {
        return fmt.Errorf("value must be array for byKey")
    }

    for _, v := range valArr {
        vm, ok := v.(map[string]interface{})

```

```

        if !ok {
            continue
        }
        k, _ := vm[p.ByKey]
        found := false
        for _, item := range arr {
            im, _ := item.(map[string]interface{})
            if im[p.ByKey] == k {
                deepMerge(im, vm)
                found = true
                break
            }
        }
        if !found {
            arr = append(arr, vm)
        }
    }
    return nil
}

// ---- scalar 数组(itemOps) ----
func applyItemOps(existing interface{}, p PatchOp) error {
    arr, ok := existing.([]interface{})
    if !ok {
        return fmt.Errorf("itemOps requires array")
    }
    vals := toSlice(p.Value)

    switch p.ItemOps {
    case "add":
        for _, v := range vals {
            if !contains(arr, v) {
                arr = append(arr, v)
            }
        }
    case "remove":
        arr = filter(arr, func(v interface{}) bool {
            return !contains(vals, v)
        })
    case "replace":
        if p.Old == nil {
            return fmt.Errorf("itemOps replace requires 'old' field")
        }
        for i, v := range arr {
            if reflect.DeepEqual(v, p.Old) {

```

```

        arr[i] = p.Value
        break
    }

    }

case "keep":
    arr = filter(arr, func(v interface{}) bool {
        return contains(vals, v)
    })

case "disable":
    arr = filter(arr, func(v interface{}) bool {
        return !contains(vals, v)
    })

default:
    return fmt.Errorf("unknown itemOps: %s", p.ItemOps)
}

return nil
}

// ---- deepMerge ----
func deepMerge(dst, src map[string]interface{}) {
    for k, sv := range src {
        dv, exists := dst[k]
        if !exists {
            dst[k] = sv
            continue
        }
        if isMap(dv) && isMap(sv) {
            deepMerge(dv.(map[string]interface{}), sv.(map[string]interface{}))
            continue
        }
        // slice 直接覆盖
        if isSlice(dv) && isSlice(sv) {
            dst[k] = sv
            continue
        }
        // scalar 覆盖
        dst[k] = sv
    }
}

```

六、diff.go (dry-run + diff)

```

package yamlpatch

import (
    "reflect"
    "strings"
)

// DryRun 返回应用 patch 后的新对象 (不修改原始对象)
func DryRun(obj map[string]interface{}, patches []PatchOp) (map[string]interface{}, error) {
    clone := cloneMap(obj)
    return clone, Apply(clone, patches)
}

// DiffMaps 比较两个 map, 返回结构化 diff
func DiffMaps(before, after map[string]interface{}) []Diff {
    var diffs []Diff
    diffRecursive(&diffs, before, after, "")
    return diffs
}

func diffRecursive(diffs *[]Diff, b, a map[string]interface{}, path string) {
    for k, av := range a {
        bv, ok := b[k]
        fullPath := joinPath(path, k)

        if !ok {
            *diffs = append(*diffs, Diff{Path: fullPath, Old: nil, New: av})
            continue
        }
        if !reflect.DeepEqual(bv, av) {
            if isMap(bv) && isMap(av) {
                diffRecursive(diffs, bv.(map[string]interface{}), av.(map[string]interface{}), fullPath)
            } else {
                *diffs = append(*diffs, Diff{Path: fullPath, Old: bv, New: av})
            }
        }
    }
    // 检查被删除的 key
    for k := range b {
        if _, ok := a[k]; !ok {
            fullPath := joinPath(path, k)

```

```

                                *diffs = append(*diffs, Diff{Path: fullPath, Old: b[k], New: ni
1}))
                                }
                                }
                                }
                                }

```

七、utils.go

```

package yamlpatch

import (
    "reflect"
    "strings"
)

func isMap(v interface{}) bool {
    _, ok := v.(map[string]interface{})
    return ok
}

func isSlice(v interface{}) bool {
    _, ok := v.([]interface{})
    return ok
}

func contains(arr []interface{}, v interface{}) bool {
    for _, a := range arr {
        if reflect.DeepEqual(a, v) {
            return true
        }
    }
    return false
}

func filter(arr []interface{}, fn func(interface{}) bool) []interface{} {
    out := make([]interface{}, 0, len(arr))
    for _, v := range arr {
        if fn(v) {
            out = append(out, v)
        }
    }
    return out
}

```



```

func toSlice(v interface{}) []interface{} {
    if s, ok := v.([]interface{}); ok {
        return s
    }
    return []interface{}{v}
}

func joinPath(prefix, key string) string {
    if prefix == "" {
        return "/" + key
    }
    return prefix + "/" + key
}

// cloneMap 深拷贝
func cloneMap(m map[string]interface{}) map[string]interface{} {
    out := make(map[string]interface{})
    for k, v := range m {
        switch val := v.(type) {
        case map[string]interface{}:
            out[k] = cloneMap(val)
        case []interface{}:
            out[k] = cloneSlice(val)
        default:
            out[k] = v
        }
    }
    return out
}

func cloneSlice(s []interface{}) []interface{} {
    out := make([]interface{}, len(s))
    for i, v := range s {
        switch val := v.(type) {
        case map[string]interface{}:
            out[i] = cloneMap(val)
        case []interface{}:
            out[i] = cloneSlice(val)
        default:
            out[i] = v
        }
    }
    return out
}

```

八、使用示例

```
package main

import (
    "fmt"
    "github.com/your-org/yamlpatch"
    "gopkg.in/yaml.v3"
)

func main() {
    // 加载 YAML
    var obj map[string]interface{}
    yaml.Unmarshal([]byte(`
spec:
  components:
    - name: web
      image: nginx:1.20
  tags:
    - dev
    - legacy
`), &obj)

    // 构造 Patch
    patches := []yamlpatch.PatchOp{
        {
            Ope:      "merge",
            PathKey:  "/spec/components",
            ByKey:    "name",
            Value: []interface{}{
                map[string]interface{}{
                    "name": "web",
                    "image": "nginx:1.25",
                },
            },
        },
        {
            Ope:      "merge",
            PathKey:  "/spec/tags",
            ItemOps:  "replace",
            Old:      "legacy",
            Value:    "production",
        },
    },
}
```

```
    }

    // dry-run
    after, _ := yamlpatch.DryRun(obj, patches)
    diffs := yamlpatch.DiffMaps(obj, after)
    for _, d := range diffs {
        fmt.Printf("%s: -%v +%v\n", d.Path, d.Old, d.New)
    }
}
```

九、你现在拥有的完整能力

能力	状态
struct 数组 byKey	
scalar 数组 itemOps	
mixed 数组 replace	
old 字段替换	
disable 禁用	
deepMerge	
diff / dry-run	
独立 Go Package	

下一步

-  生成 CRD YAML (OpenAPI Schema)
-  接入 controller-runtime
-  给 IAM / RBAC Patch 示例
-  补 table-driven 单元测试

你直接说一句：

 "我要 CRD / Controller / 单测"

(注：文档部分内容可能由 AI 生成)